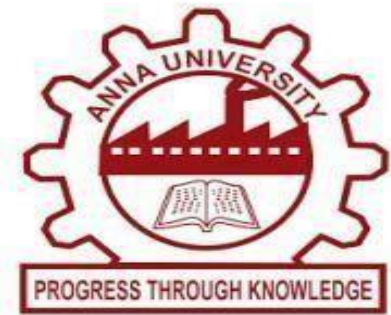




**PANIMALAR
ENGINEERING COLLEGE**
An Autonomous Institution, Affiliated to Anna University, Chennai
A Christian Minority Institution
(JAISAKTHI EDUCATIONAL TRUST)
Approved by All India Council for Technical Education



Department of Computer Science and Engineering

SAFE DRIVE: REAL TIME DROWSINESS DETECTION

Team Members:

BRIGITTE MARY A (211423104093)

DHARSHINI A (211423104130)

Batch No: A25

Guide Name:

ALIMA BEEVI A , M.E.,
(Assistant Professor)

AGENDA:

- Abstract
- Introduction
- Objective
- Literature Survey
- Problem Statement
- System Architecture
- UML Diagrams
- Modules & Methodology
- Testing & Results
- Conclusion & Future Work

ABSTRACT:

- Driver drowsiness is a leading cause of road accidents so this project focuses on
- Real-time detection of driver using facial features.
- Capture eye and mouth movements via webcam then Calculates EAR and MAR from facial landmarks
- Triggers alert when drowsiness is detected.
- Uses Python, OpenCV, and Mediapipe libraries.
- Provides a low-cost, software-based solution to prevent accidents.
- Enhances road safety through early fatigue detection.

INTRODUCTION:

- Road accidents caused by driver fatigue are rapidly increasing, highlighting the urgent need for intelligent monitoring systems.
- Real-time driver monitoring ensures safety by detecting early signs of fatigue, such as prolonged eye closure or yawning.
- The system uses computer vision and facial landmark detection, offering a non-intrusive method to analyze driver alertness.
- By tracking eye and mouth movements accurately, it identifies drowsiness and activates timely alerts.
- Designed to be cost-effective and software-based, it can be easily integrated into existing vehicles.

OBJECTIVE:

- To develop a real-time driver drowsiness detection system that monitors facial features using a standard webcam.
- To calculate Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) for identifying signs of sleepiness or yawning.
- To provide immediate alerts through sound or visual signals when drowsiness is detected during driving.
- To ensure high accuracy and reliability under different lighting and environmental conditions.
- To design a low-cost, software-only solution that enhances road safety and prevents accidents.

1. Multimodal Driver Drowsiness Detection From Video Frames

ASPECT	DETAILS
Author	P. K. Singh, A. Gupta, M. Upadhyay, A. Jain, et al.
Summary	Uses a Multimodal approach to detect drowsiness by simultaneously analyzing features (like eye and mouth movements) extracted from video frames.
Relevance	Addresses the limitation of single-feature systems by combining multiple indicators for more robust and reliable detection.
Gaps	Performance metrics need thorough comparison against real-time, lightweight single-modal systems for embedded use.
Impact	Proposes a comprehensive framework that leverages the synergy of multiple visual cues to significantly improve detection accuracy.

2.IoT-Based Non-Intrusive Automated Driver Drowsiness Monitoring Framework

ASPECT	DETAILS
Author	M. A. Khan, T. Nawaz, U. S. Khan, A. Hamza, et al.
Summary	Develops an IoT-based framework for non-intrusive, automated monitoring of driver drowsiness, focusing on logistics and public transport.
Relevance	Highly relevant for commercial applications where real-time remote monitoring and data logging are essential for fleet safety management.
Gaps	The real-time performance and scalability of the IoT communication layer under varying network conditions need detailed testing.
Impact	Provides a complete system architecture for integrating DDD into the broader ecosystem of connected vehicles and smart transportation.

3. Novel Transfer Learning Approach for Driver Drowsiness Detection

ASPECT	DETAILS
Author	H. A. Madni, A. Raza, L. Abualigah, R. Sehar, et al
DSC Relevance	Minimizes the need for massive, application-specific training datasets, speeding up development and deployment.
Gaps	The domain adaptation challenge—how well the pre-trained model performs on unique in-car video data—requires extensive validation.
Impact	Demonstrates the effectiveness of leveraging existing computational resources (pre-trained networks) to achieve high accuracy in a specialized task.

4. Driver Drowsiness Detection Using Swin Transformer and Diffusion Models (Ref. [1])

ASPECT	DETAILS
Author	S. Abd El-Nabi, A. F. Ibrahim, E.-S. M. El-Rabai, et al
Summary	Utilizes the Swin Transformer (a vision transformer) combined with Diffusion Models for robust image denoising to detect driver drowsiness.
Relevance	Introduces state-of-the-art transformer and generative models to enhance detection accuracy, especially in noisy or low-quality video feeds.
Gaps	The computational complexity of diffusion models and transformers can be a challenge for real-time deployment on resource-constrained embedded systems.
Impact	Pushes the boundary of detection reliability by integrating advanced computer vision models to solve common real-world image quality issues.

5. A Systematic Review on Driver Drowsiness Detection Using Eye Activity Measures (Ref. [8])

ASPECT	DETAILS
Author	A. Kolus
Summary	A Systematic Review that analyzes and summarizes the efficacy of various Eye Activity Measures (e.g., EAR, PERCLOS) for use in driver drowsiness detection systems.
Relevance	Provides a critical, consolidated overview of the most fundamental and widely-used vision-based techniques, establishing best practices and performance benchmarks.
Gaps	While summarizing methods, the review may not detail novel real-time implementation challenges of the latest deep learning models.
Impact	Serves as an essential foundational reference for researchers designing or comparing new vision-based DDD systems centered on eye state.

6.Driver Drowsiness Detection Based on Respiratory Signal Analysis (Ref. [4])

ASPECT	DETAILS
Author	F. Guede-Fernández, M. Fernández-Chimeno, J. Ramos-Castro, et al.
Summary	Proposes a non-vision method to detect drowsiness by analyzing the driver's Respiratory Signal (breathing patterns), captured via external sensors.
Relevance	Offers a valuable non-intrusive alternative to vision-based methods, which can be affected by lighting or obstructions like sunglasses.
Gaps	Requires dedicated, often specialized, sensors (e.g., on the seat or seatbelt), introducing complexity and cost compared to a simple webcam.
Impact	Contributes to multimodal research by validating a reliable physiological indicator that can be integrated with visual data for highly robust detection.

7.Instruction Set Extension of a RISC-V Based SoC for Driver Drowsiness Detection (Ref. [9], [11])

ASPECT	DETAILS
Author	S. K. Mousavikia, S. B. Ors Yalcin, E. Gholizadehazari, et al.
Summary	Focuses on hardware acceleration by extending the instruction set of a RISC-V System-on-Chip (SoC) to efficiently execute drowsiness detection algorithms.
Relevance	Crucial for the commercial viability and real-time performance of DDD systems in automotive embedded devices by ensuring high speed and low power consumption.
Gaps	The solution is highly hardware-specific, meaning the code and performance benefits are tied directly to the customized RISC-V architecture.
Impact	Demonstrates a path toward specialized edge computing and hardware/software co-design to enable the deployment of complex deep learning models in vehicles.

8.Instruction Set Extension of a RISC-V Based SoC for Driver Drowsiness Detection (Ref. [9])

ASPECT	DETAILS
Author	S. K. Mousavikia, S. B. Ors Yalcin, E. Gholizadehazari, and M. Mousazadeh
Summary	Focuses on optimizing hardware by creating an Instruction Set Extension (ISE) for a RISC-V System-on-Chip (SoC) to efficiently execute DDD algorithms.
Relevance	Addresses the crucial need for high-performance, low-power execution of deep learning models on embedded automotive hardware.
Gaps	The solution is highly specific to the customized RISC-V architecture, limiting its direct portability to other processor platforms.
Impact	Provides a methodology for hardware/software co-design, demonstrating significant speedup and efficiency for real-time in-vehicle processing.

9.Facial Feature-Based Drowsiness Detection With Multi-Scale Convolutional Neural Network (Ref. [10])

ASPECT	DETAILS
Author	V. Vijaypriya and M. Uma
Summary	Proposes a drowsiness detection method based on Facial Features using a Multi-Scale Convolutional Neural Network (MS-CNN) architecture.
Relevance	The MS-CNN allows the system to capture both fine-grained and coarse features of the face, enhancing the robustness of facial landmark detection for calculating fatigue metrics.
Gaps	The performance under extreme head poses or significant occlusion (e.g., from hands or large accessories) is often a constraint for pure facial feature models.
Impact	Contributes a strong CNN-based approach that emphasizes the importance of scale invariance in recognizing subtle, multi-level visual cues of drowsiness.

10.Instruction Set Extension of a RISC-V Based SoC for Driver Drowsiness Detection (Ref. [11])

ASPECT	DETAILS
Author	S. K. Mousavikia, S. B. Ors Yalcin, E. Gholizadehazari, and M. Mousazadeh
Summary	The same study as [9]. It describes the implementation of an Instruction Set Extension (ISE) on a RISC-V SoC to accelerate the computational tasks associated with Driver Drowsiness Detection.
Relevance	Highlights the current trend in research toward domain-specific hardware to meet the stringent power and latency requirements of safety-critical automotive systems.
Gaps	Requires specialized knowledge and tools for hardware modification (ISA extension) rather than purely software optimization.
Impact	Successfully integrates algorithm design with processor architecture, paving the way for efficient deployment of complex AI models at the edge.

PROBLEM STATEMENT:

- Driver drowsiness is one of the leading causes of road accidents, yet it's often unnoticed until a crash occurs.
- There is no built-in drowsiness detection system in most private and commercial vehicles.
- Manual detection is unreliable—we cannot depend on drivers or others to notice fatigue.
- In critical moments, every second counts—a delay in alerting can cost lives.

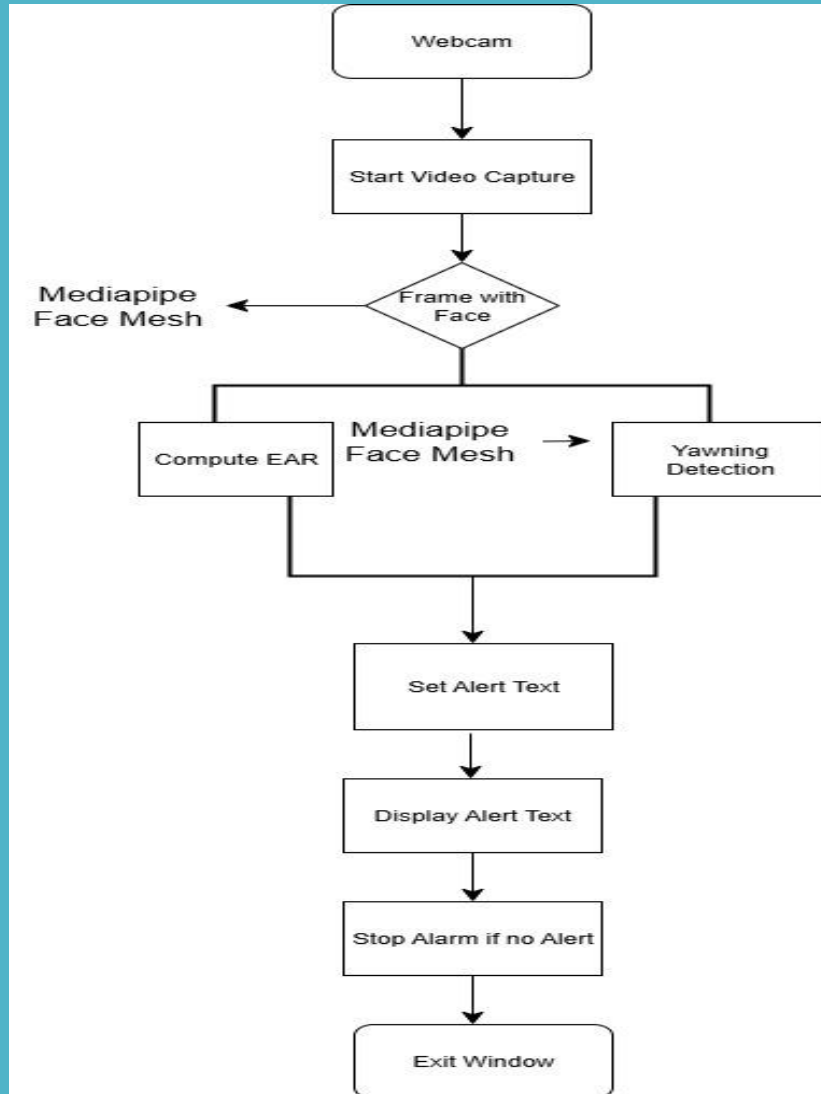
A real-time, automated system is needed to:

Continuously monitor Eyes and Mouth

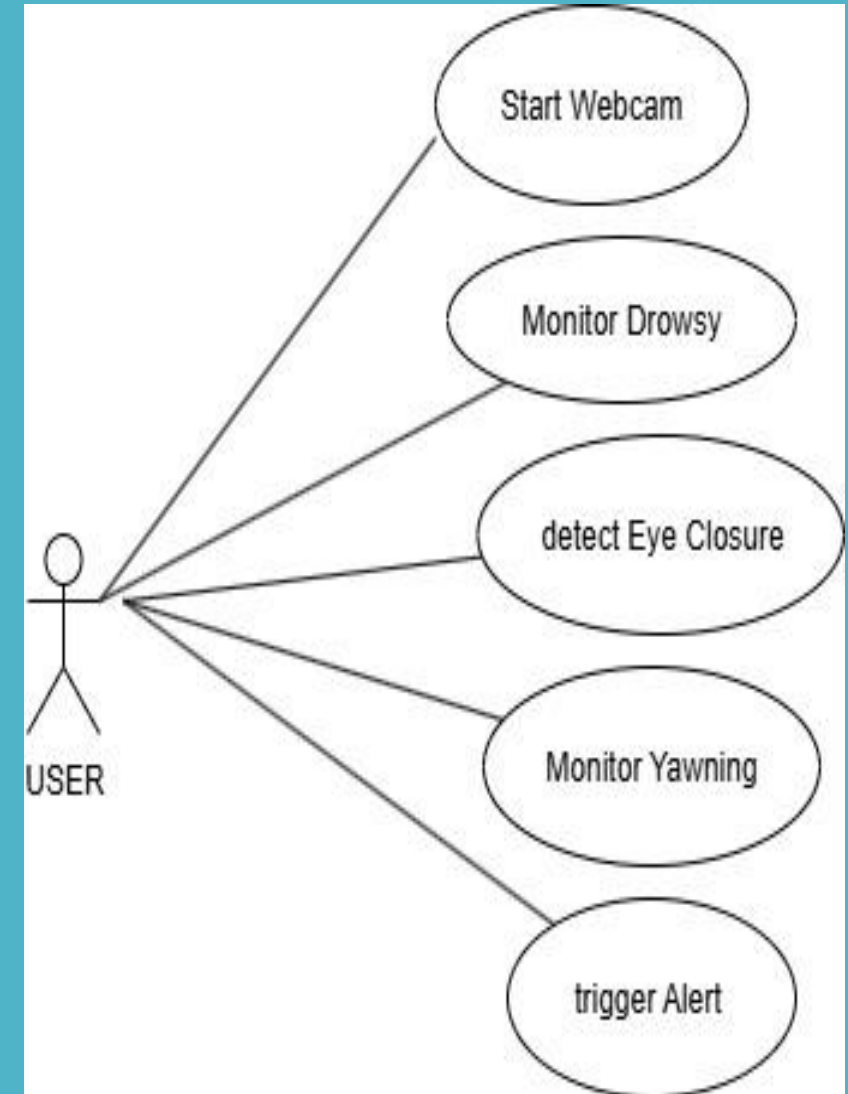
Detect early signs of sleepiness (like eye closure or yawning)

Alert the driver immediately through sound, vibration, or other warnings

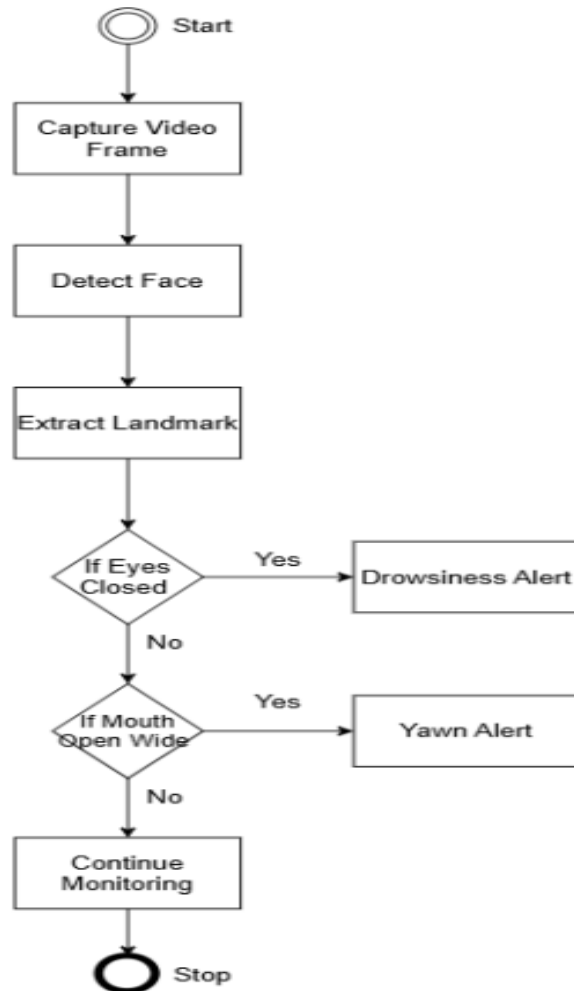
ARCHITECTURE DIGRAM:



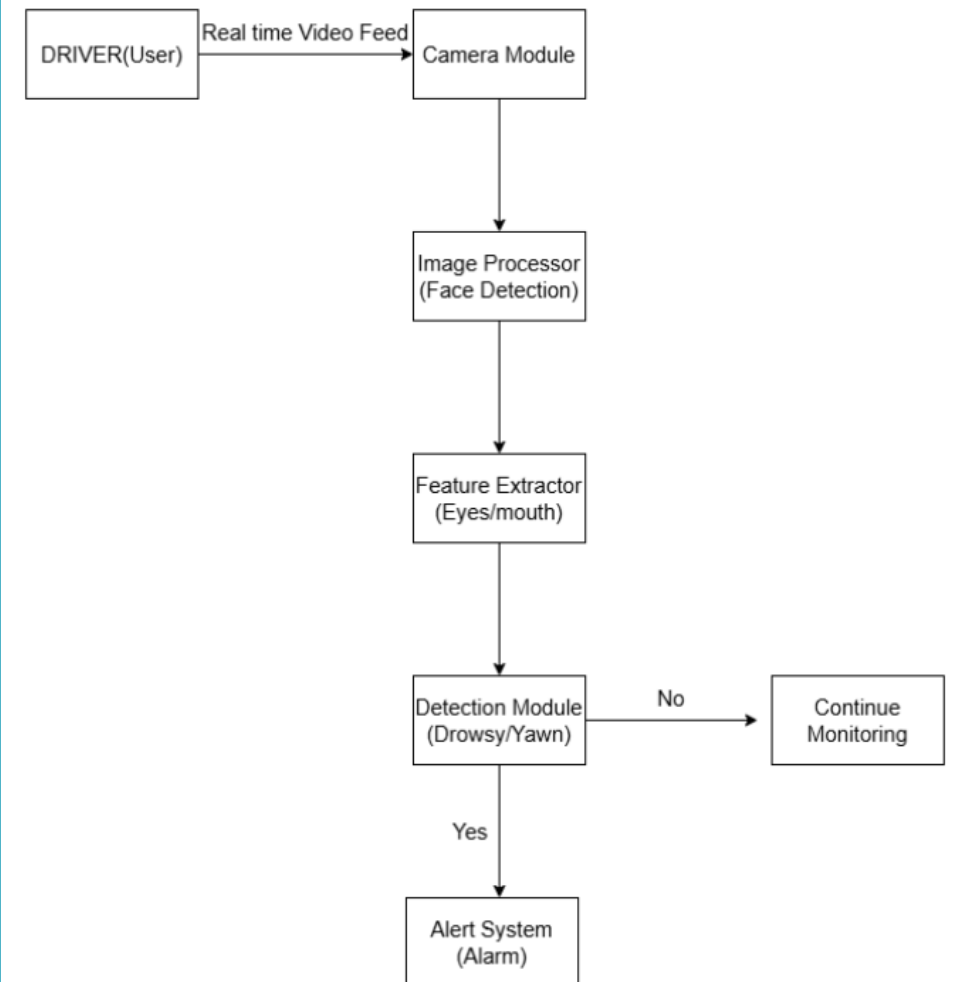
USE CASE DIGRAM:



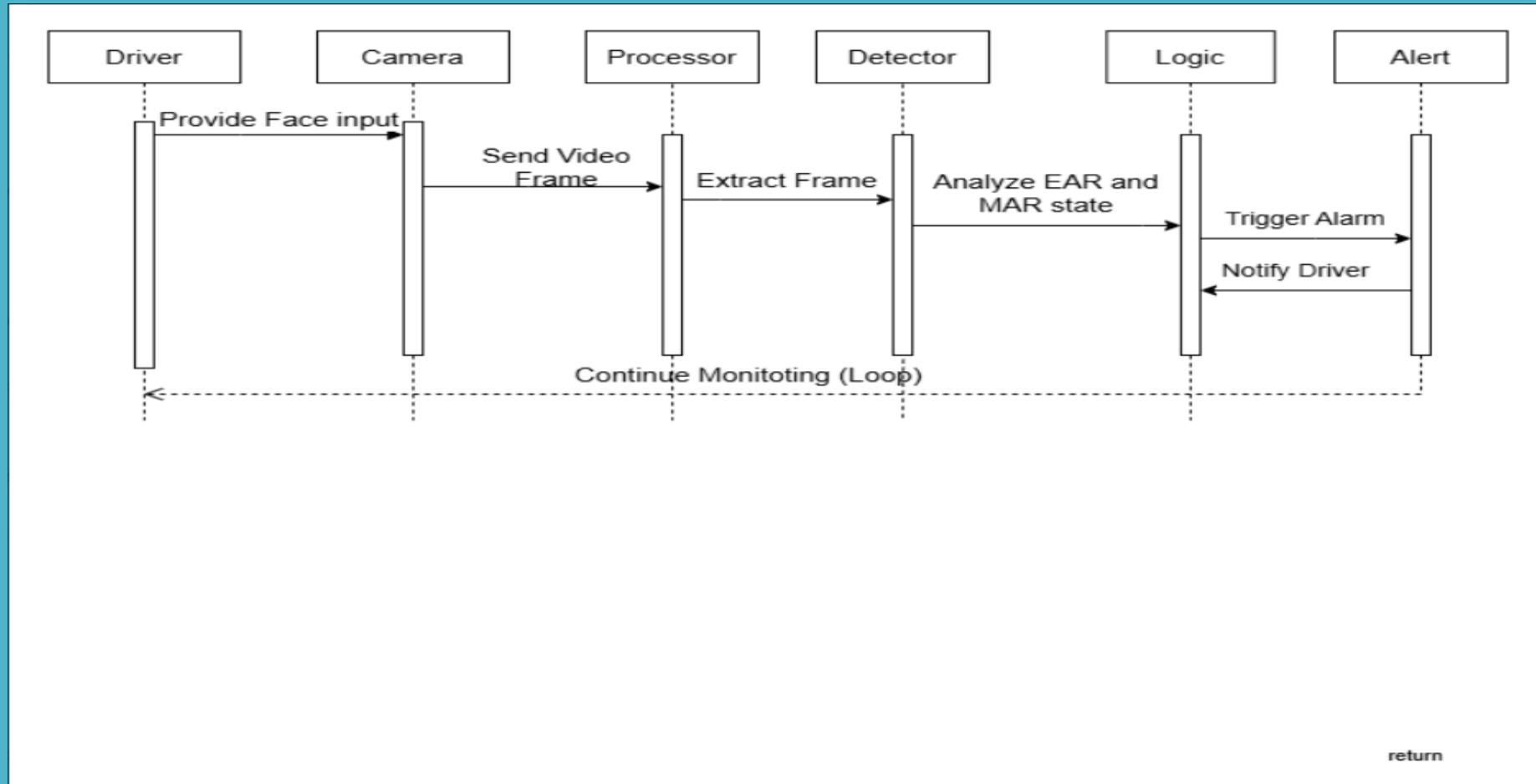
ACTIVITY DIGRAM:



DATA FLOW DIGRAM(L-1) :



SEQUENCE DIGRAM:



MODULES:

1. Image Capture Module	Captures continuous video frames from the webcam to monitor the driver's face in real time.
2. Face & Landmark Detection Module	Detects the face region and identifies key facial landmarks like eyes and mouth using Mediapipe / Dlib.
3. Feature Extraction Module	Calculates Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) from facial landmarks for analysis.
4. Drowsiness Detection Module	Analyzes EAR and MAR values to detect eye closure or yawning, indicating possible drowsiness.
5. Alert Generation Module	Activates a buzzer or visual alert instantly when fatigue symptoms are detected to wake the driver.
6. Performance Evaluation Module	Tests system accuracy, response time, and reliability under various lighting and real-world conditions.

ALGORITHM:

Step 1: Video Input Acquisition

The system captures continuous video frames of the driver's face using a webcam.

Step 2: Face and Landmark Detection

Mediapipe or Dlib detects facial landmarks such as eyes and mouth for further analysis.

Step 3: EAR and MAR Calculation

The **Eye Aspect Ratio (EAR)** and **Mouth Aspect Ratio (MAR)** are computed from landmark coordinates.

Step 4: Threshold Evaluation

The calculated EAR and MAR values are compared against predefined thresholds to detect fatigue signs.

ALGORITHM:

Step 5: Drowsiness Decision Logic

If the EAR falls below or MAR rises above the threshold for a certain duration, drowsiness is confirmed.

Step 6: Alert Activation

A buzzer or alarm is triggered immediately to alert the driver and prevent potential accidents.

Step 7: Performance Monitoring

The system continuously evaluates detection accuracy and adjusts thresholds for reliable performance.

TESTING:

Unit Testing	Each individual module, such as face detection and alert generation, is tested to ensure correct functionality.
Integration Testing	Combined modules are tested together to verify smooth data flow between image capture and alert system.
System Testing	The complete application is evaluated under real-time conditions to ensure overall system stability.
Performance Testing	EAR and MAR values are tested under varied lighting and face angles to measure detection speed and accuracy.
Validation Testing	The system output is compared with actual driver states to confirm reliable drowsiness detection.
User Testing	Real users operate the system in simulated driving environments to evaluate practical usability and alert timing.

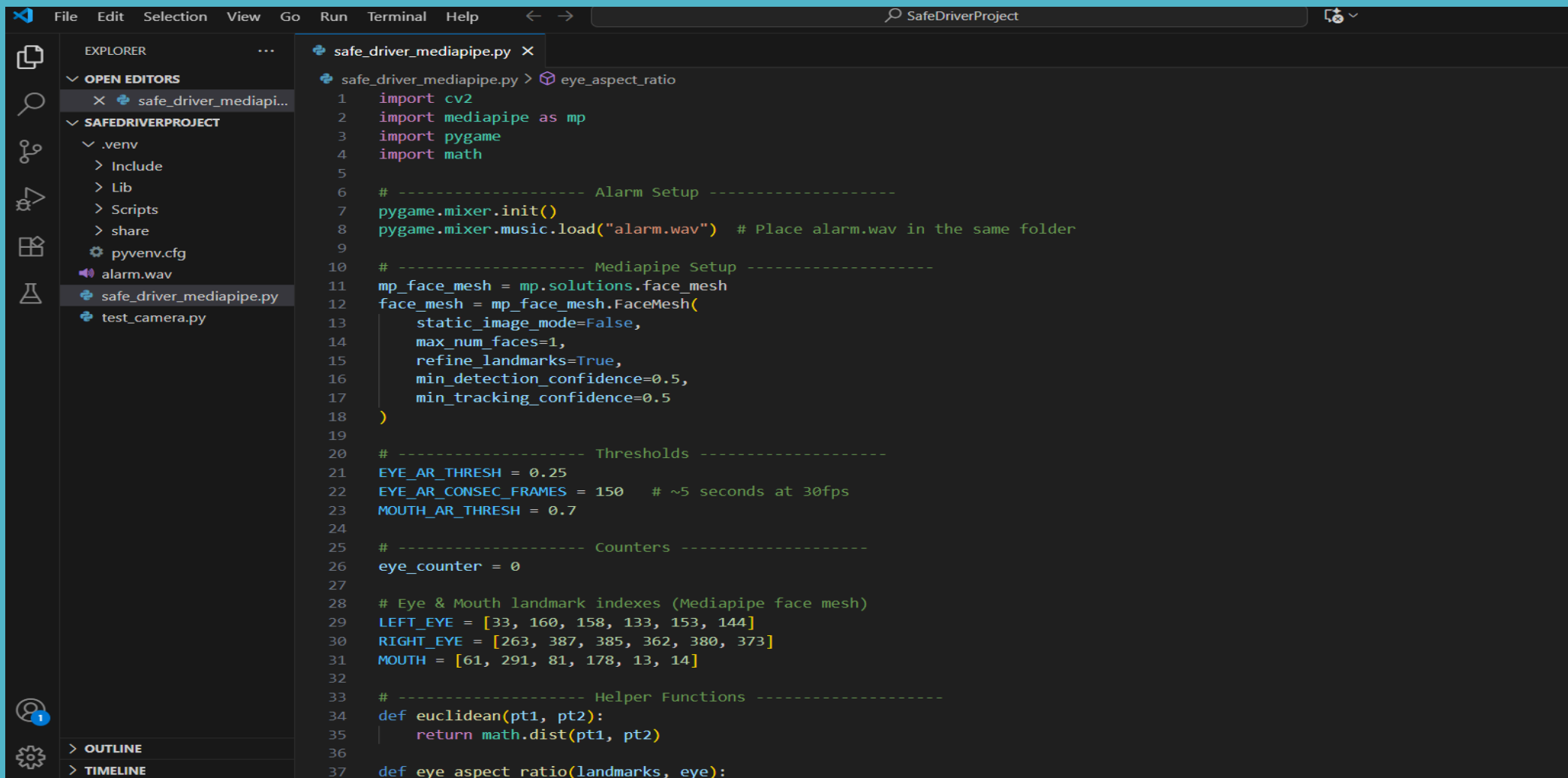
TEST CASES AND VALIDATION:

Test Case ID	Input Condition	Expected Output	Result
TC01	Eyes closed > 2 seconds	Buzzer alert activated	Pass
TC02	Eyes open normally	No alert generated	Pass
TC03	Yawning detected	Warning sound triggered	Pass
TC04	Poor lighting condition	Face detection still accurate	Pass
TC05	Normal lighting, alert state	System remains inactive	Pass

PERFORMANCE ANALYSIS:

- The system was tested under different lighting and facial orientations to evaluate accuracy, response time, and reliability.
- Achieved an average accuracy of around 95% in detecting eye closure and yawning using EAR and MAR values.
- The response time was less than one second, ensuring real-time alert generation for driver safety.
- The system maintained stable performance even under low-light and slight head movement conditions.
- False alarms were reduced by optimizing threshold values, improving both precision and system consistency.
- Overall, the project demonstrated efficient and robust detection, suitable for real-world driving environments.

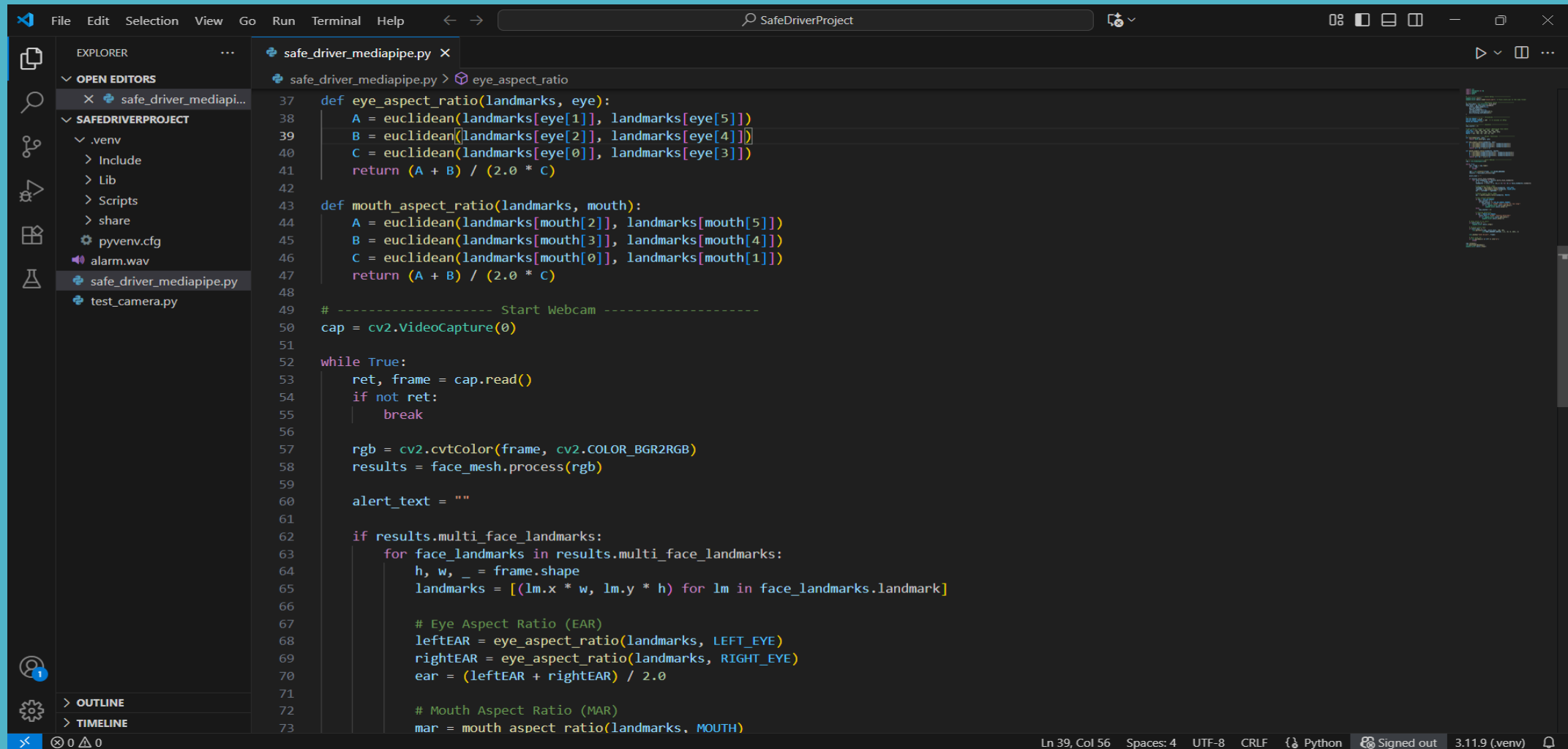
EXPERIMENTAL RESULTS:



The image shows a screenshot of the Visual Studio Code editor interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The Explorer sidebar on the left shows the project structure for 'SafeDriverProject', including folders like '.venv', 'Include', 'Lib', 'Scripts', 'share', and files like 'pyvenv.cfg', 'alarm.wav', 'safe_driver_mediapipe.py', and 'test_camera.py'. The main editor window displays the code for 'safe_driver_mediapipe.py', which is currently focused on the 'eye_aspect_ratio' function. The code includes imports for cv2, mediapipe, pygame, and math. It features sections for Alarm Setup, Mediapipe Setup, Thresholds, and Counters. The 'eye_aspect_ratio' function is defined at the bottom, taking 'landmarks' and 'eye' as arguments.

```
safe_driver_mediapipe.py > eye_aspect_ratio
1  import cv2
2  import mediapipe as mp
3  import pygame
4  import math
5
6  # ----- Alarm Setup -----
7  pygame.mixer.init()
8  pygame.mixer.music.load("alarm.wav") # Place alarm.wav in the same folder
9
10 # ----- Mediapipe Setup -----
11 mp_face_mesh = mp.solutions.face_mesh
12 face_mesh = mp_face_mesh.FaceMesh(
13     static_image_mode=False,
14     max_num_faces=1,
15     refine_landmarks=True,
16     min_detection_confidence=0.5,
17     min_tracking_confidence=0.5
18 )
19
20 # ----- Thresholds -----
21 EYE_AR_THRESH = 0.25
22 EYE_AR_CONSEC_FRAMES = 150 # ~5 seconds at 30fps
23 MOUTH_AR_THRESH = 0.7
24
25 # ----- Counters -----
26 eye_counter = 0
27
28 # Eye & Mouth landmark indexes (Mediapipe face mesh)
29 LEFT_EYE = [33, 160, 158, 133, 153, 144]
30 RIGHT_EYE = [263, 387, 385, 362, 380, 373]
31 MOUTH = [61, 291, 81, 178, 13, 14]
32
33 # ----- Helper Functions -----
34 def euclidean(pt1, pt2):
35     return math.dist(pt1, pt2)
36
37 def eye_aspect_ratio(landmarks, eye):
```

EXPERIMENTAL RESULTS:



The image shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left shows a project named 'SAFEDRIVERPROJECT' with files like '.venv', 'pyvenv.cfg', 'alarm.wav', 'safe_driver_mediapipe.py', and 'test_camera.py'. The main editor area displays the 'safe_driver_mediapipe.py' file. The code defines two functions, 'eye_aspect_ratio' and 'mouth_aspect_ratio', which calculate the Euclidean distance between facial landmarks to determine the aspect ratio of the eyes and mouth. The main script starts a webcam capture loop, reads frames, and processes them using the 'face_mesh' library to detect landmarks and calculate these ratios. The status bar at the bottom indicates the current position is Line 39, Column 56, with 4 spaces, UTF-8 encoding, CRLF line endings, Python interpreter, and a signed-out state.

```
File Edit Selection View Go Run Terminal Help
SafeDriverProject

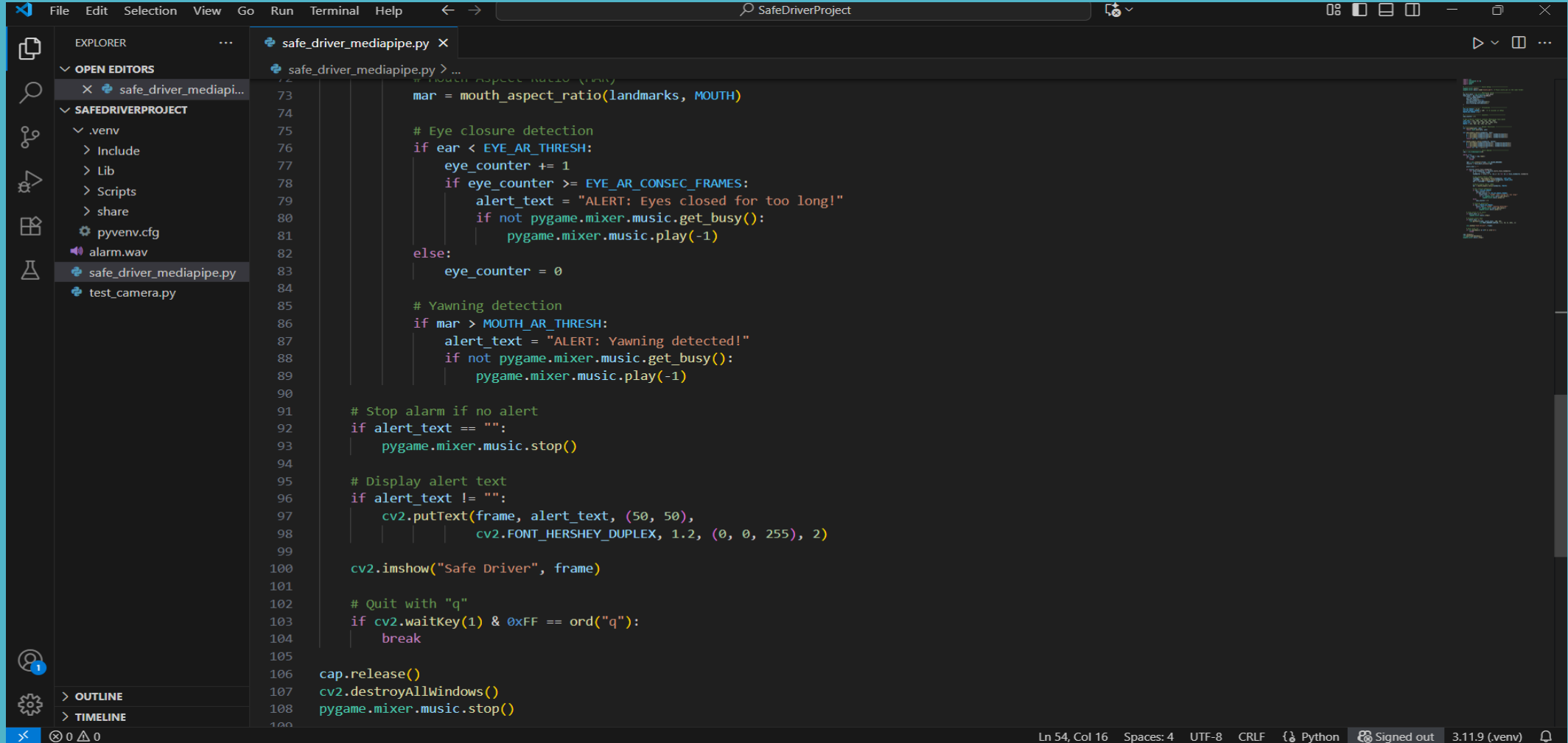
EXPLORER
OPEN EDITORS
safe_driver_mediapi...
SAFEDRIVERPROJECT
.venv
  Include
  Lib
  Scripts
  share
pyvenv.cfg
alarm.wav
safe_driver_mediapipe.py
test_camera.py

safe_driver_mediapipe.py
safe_driver_mediapipe.py > eye_aspect_ratio

37 def eye_aspect_ratio(landmarks, eye):
38     A = euclidean(landmarks[eye[1]], landmarks[eye[5]])
39     B = euclidean(landmarks[eye[2]], landmarks[eye[4]])
40     C = euclidean(landmarks[eye[0]], landmarks[eye[3]])
41     return (A + B) / (2.0 * C)
42
43 def mouth_aspect_ratio(landmarks, mouth):
44     A = euclidean(landmarks[mouth[2]], landmarks[mouth[5]])
45     B = euclidean(landmarks[mouth[3]], landmarks[mouth[4]])
46     C = euclidean(landmarks[mouth[0]], landmarks[mouth[1]])
47     return (A + B) / (2.0 * C)
48
49 # ----- Start Webcam -----
50 cap = cv2.VideoCapture(0)
51
52 while True:
53     ret, frame = cap.read()
54     if not ret:
55         break
56
57     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
58     results = face_mesh.process(rgb)
59
60     alert_text = ""
61
62     if results.multi_face_landmarks:
63         for face_landmarks in results.multi_face_landmarks:
64             h, w, _ = frame.shape
65             landmarks = [(lm.x * w, lm.y * h) for lm in face_landmarks.landmark]
66
67             # Eye Aspect Ratio (EAR)
68             leftEAR = eye_aspect_ratio(landmarks, LEFT_EYE)
69             rightEAR = eye_aspect_ratio(landmarks, RIGHT_EYE)
70             ear = (leftEAR + rightEAR) / 2.0
71
72             # Mouth Aspect Ratio (MAR)
73             mar = mouth_aspect_ratio(landmarks, MOUTH)
```

Ln 39, Col 56 Spaces: 4 UTF-8 CRLF Python Signed out 3.11.9 (.venv)

EXPERIMENTAL RESULTS:

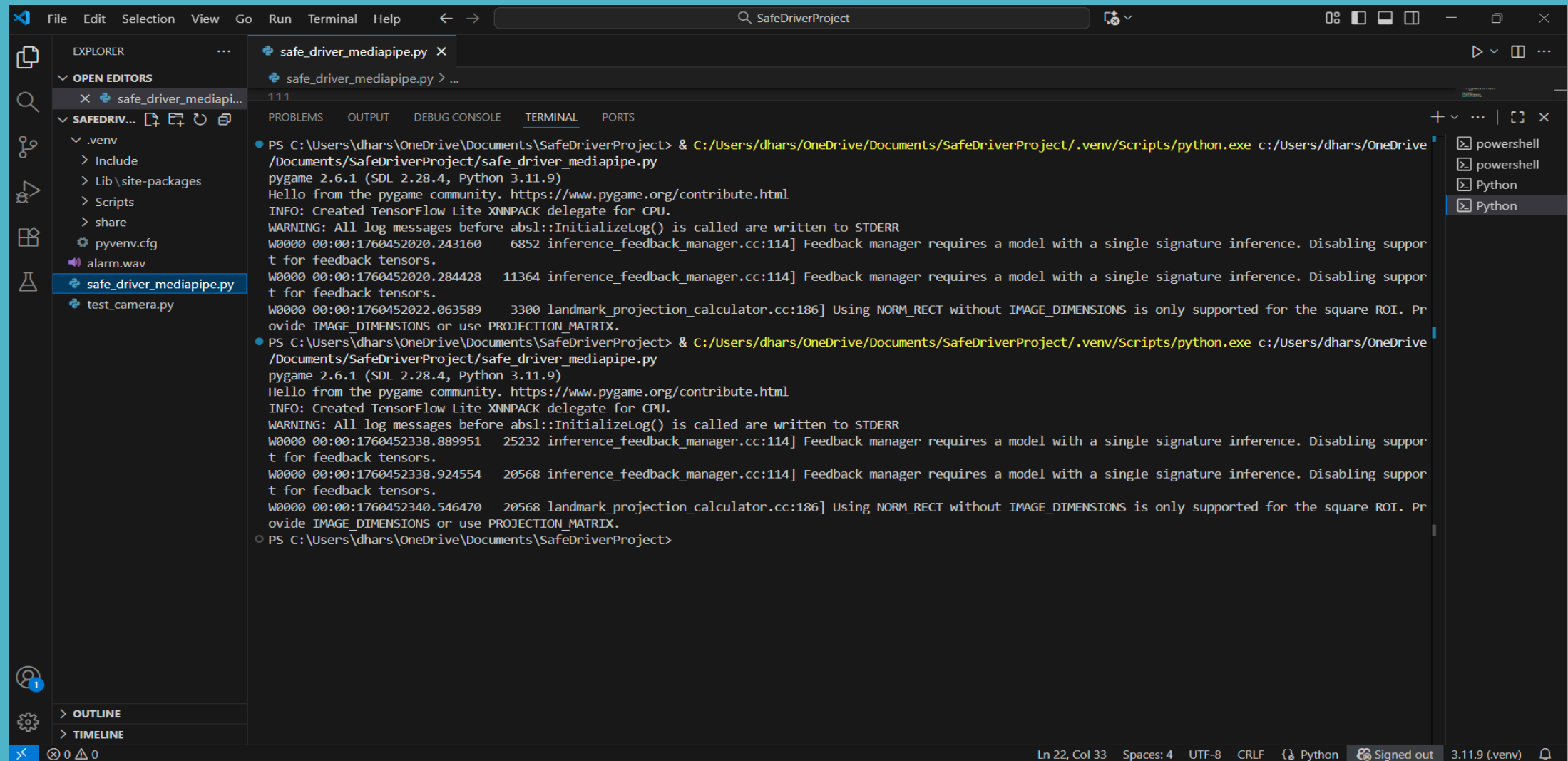


The screenshot displays the Visual Studio Code interface with a Python file named `safe_driver_mediapipe.py` open. The file is part of a project named `SafeDriverProject`. The code implements a driver safety system using MediaPipe for facial landmark detection, OpenCV for image processing, and Pygame for audio alerts. It features eye closure detection and yawning detection, both triggering an alarm and displaying an alert message on the screen.

```
73     mar = mouth_aspect_ratio(landmarks, MOUTH)
74
75     # Eye closure detection
76     if ear < EYE_AR_THRESH:
77         eye_counter += 1
78         if eye_counter >= EYE_AR_CONSEC_FRAMES:
79             alert_text = "ALERT: Eyes closed for too long!"
80             if not pygame.mixer.music.get_busy():
81                 pygame.mixer.music.play(-1)
82         else:
83             eye_counter = 0
84
85     # Yawning detection
86     if mar > MOUTH_AR_THRESH:
87         alert_text = "ALERT: Yawning detected!"
88         if not pygame.mixer.music.get_busy():
89             pygame.mixer.music.play(-1)
90
91     # Stop alarm if no alert
92     if alert_text == "":
93         pygame.mixer.music.stop()
94
95     # Display alert text
96     if alert_text != "":
97         cv2.putText(frame, alert_text, (50, 50),
98                     cv2.FONT_HERSHEY_DUPLEX, 1.2, (0, 0, 255), 2)
99
100    cv2.imshow("Safe Driver", frame)
101
102    # Quit with "q"
103    if cv2.waitKey(1) & 0xFF == ord("q"):
104        break
105
106    cap.release()
107    cv2.destroyAllWindows()
108    pygame.mixer.music.stop()
109
```

The interface includes a sidebar with the Explorer, Open Editors, and Outline/Timeline views. The Explorer shows the project structure with files like `.venv`, `Include`, `Lib`, `Scripts`, `share`, `pyvenv.cfg`, `alarm.wav`, `safe_driver_mediapipe.py`, and `test_camera.py`. The status bar at the bottom indicates the current line and column (Ln 54, Col 16), encoding (UTF-8), and other settings.

EXPERIMENTAL RESULTS:



The screenshot displays the Visual Studio Code interface with a project named 'SafeDriverProject'. The Explorer sidebar on the left shows the file structure, including a '.venv' directory and a 'safe_driver_mediapipe.py' file. The main editor window shows the content of 'safe_driver_mediapipe.py', which is a Python script. The Terminal panel at the bottom shows the output of the script, which includes a series of log messages and warnings. The status bar at the bottom indicates the current line and column (Ln 22, Col 33) and the active Python environment (3.11.9 (venv)).

```
111
PS C:\Users\dhars\OneDrive\Documents\SafeDriverProject> & C:/Users/dhars/OneDrive/Documents/SafeDriverProject/.venv/Scripts/python.exe c:/Users/dhars/OneDrive/Documents/SafeDriverProject/safe_driver_mediapipe.py
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1760452020.243160      6852 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
W0000 00:00:1760452020.284428      11364 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
W0000 00:00:1760452022.063589      3300 landmark_projection_calculator.cc:186] Using NORM_RECT without IMAGE_DIMENSIONS is only supported for the square ROI. Provide IMAGE_DIMENSIONS or use PROJECTION_MATRIX.
PS C:\Users\dhars\OneDrive\Documents\SafeDriverProject> & C:/Users/dhars/OneDrive/Documents/SafeDriverProject/.venv/Scripts/python.exe c:/Users/dhars/OneDrive/Documents/SafeDriverProject/safe_driver_mediapipe.py
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1760452338.889951      25232 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
W0000 00:00:1760452338.924554      20568 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
W0000 00:00:1760452340.546470      20568 landmark_projection_calculator.cc:186] Using NORM_RECT without IMAGE_DIMENSIONS is only supported for the square ROI. Provide IMAGE_DIMENSIONS or use PROJECTION_MATRIX.
PS C:\Users\dhars\OneDrive\Documents\SafeDriverProject>
```

EXPERIMENTAL RESULTS:

Driver Drowsiness Detection



Drowsy

Drowsiness Detected

CONCLUSION:

- The system successfully detects driver drowsiness in real time using facial landmarks and aspect ratio calculations.
- It effectively identifies eye closure and yawning to determine fatigue and trigger alert mechanisms.
- The use of Python, OpenCV, and Mediapipe ensures accurate, lightweight, and efficient implementation.
- The software-based approach provides a low-cost solution that can be integrated easily into any vehicle.
- Experimental testing confirmed high accuracy and quick response, making it practical for real-world driving scenarios.
- Overall, the project achieves its goal of enhancing driver safety and reducing road accidents caused by drowsiness

FUTURE IMPLEMENTATION:

- Multi-sensor integration: Future systems could include heart rate, breathing, or EEG data along with facial cues. This would make the detection more accurate and reduce false alarms.
- IoT and edge deployment: Implement the system on low-cost IoT or edge devices for real-time monitoring in buses, trucks, and fleets.
- Cloud-based analytics: Cloud platforms could allow companies to monitor many drivers at once, helping predict and prevent accidents before they happen.
- Vehicle system integration: Future designs may link the system with vehicle safety features to trigger alerts, seat vibrations, or even support automatic control if the driver is too drowsy.
- Personalized driver profiling: The system could be trained to understand each driver's unique habits and fatigue patterns, leading to more precise detection.
- Advanced AI models: Use deep learning and Vision Transformer architectures to improve performance under poor lighting or partial face visibility

REFERENCES:

- [1] S. Abd El-Nabi, A. F. Ibrahim, E.-S. M. El-Rabai, O. F. Hassan, N. F. Soliman, K. F. Ramadan, and W. El-Shafai, "Driver Drowsiness Detection Using Swin Transformer and Diffusion Models for Robust Image Denoising," IEEE Access, vol 13, no. –, pp. 71880–71892, April 2025.
- [2] P. K. Singh, A. Gupta, M. Upadhyay, A. Jain, M. Khari, and P. S. Lamba, "Multimodal Driver Drowsiness Detection From Video Frames," Journal of Mobile Multimedia, vol. 19, no. 2, pp. 567–586, November 2022
- [3] A.Ziryawulawo, M.Kirabo, C.Mwikirize, J.Serugunda, E. Mugume, and S. P. Miyingo, "Machine Learning Based Driver Monitoring System: A Case Study for the Kayoola EVS," South African Institute of Electrical Engineers, vol. 114, no. 2, pp. 40–47, June 2023.
- [4] F. Guede-Fernández, M. Fernández-Chimeno, J. Ramos-Castro, and M. A. García-González, "Driver Drowsiness Detection Based on Respiratory Signal Analysis," IEEE Access, vol. 11, no. –, pp. 13579–13591, February 2023 [CITATION 1](file:///C:/Users/dhars/Downloads/ppt4.pdf).
- [5] M. A. Khan, T. Nawaz, U. S. Khan, A. Hamza, and N. Rashid, "IoT Based Non-Intrusive Automated Driver Drowsiness Monitoring Framework for Logistics and Public Transport Applications to Enhance Road Safety," IEEE Access, vol. 11, no. –, pp. 122345–122360, December 2023.
- [6] M. Ramzan, M. Fayyaz, A. Abid, T. J. Alahmadi, H. Nobanee, and A. Rehman, "A Novel Hybrid Approach for Driver Drowsiness Detection Using a Custom Deep Learning Model," IEEE Access, vol. 12, no. –, pp. 126866–126881, August 2024.
- [7] H. A. Madni, A. Raza, L. Abualigah, R. Sehar, and N. Thalji, "Novel Transfer Learning Approach for Driver Drowsiness Detection Using Eye Movement Behaviour," IEEE Access, vol. 12, no. –, pp. 64765–64778, May 2024.[CITATION 1](file:///C:/Users/dhars/Downloads/ppt7.pdf)
- [8] A. Kolus, "A Systematic Review on Driver Drowsiness Detection Using Eye Activity Measures," IEEE Access, vol. 12, no. –, pp. 97969–97993, July 2024.
- [9] S. K. Mousavikia, S.B.Ors Yalcin, E. Gholizadehazari, and M. Mousazadeh, "Instruction Set Extension of a Risc V Based SoC for Driver Drowsiness Detection," IEEE Access, vol. 10, no. –, pp. 58151–58162, June 2022.
- [10] V. Vijaypriya and M. Uma, "Facial Feature-Based Drowsiness Detection With Multi-Scale Convolutional Neural Network," Department of Computational Intelligence, SRM Institute of Science and Technology, Kattankulathur, India.